Davante

11

BASE DE DATOS

Consultas multitabla y subconsultas

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Consultas sobre varias tablas 5

/ 3. Composiciones internas 6

3.1. JOIN o INNER JOIN 6

3.2. NATURAL JOIN 6

/ 4. Producto cartesiano 7

/ 5. Composiciones externas 7

5.1. LEFT JOIN y NATURAL LEFT JOIN 7

5.2. RIGHT JOIN y NATURAL RIGHT JOIN 8

/ 6. FULL OUTER JOIN 8

/ 7. Subconsultas 9

7.1. Subconsultas de varias filas 10

/ 8. Subconsultas con ALL, ANY, IN y NOT IN 11

8.1. ALL y ANY 11

8.2. IN y NOT IN 11

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 9. Subconsultas con EXISTS, NOT EXISTS, FROM, JOIN, GROUP BY y HAVING 12

9.1. Subconsultas con EXISTS y NOT EXISTS 12

9.2. Subconsultas con FROM y JOIN 12

9.3. Subconsultas con GROUP BY y HAVING 13

/ 10. Operaciones de conjuntos 13

10.1. Cláusula UNION e INTERSECT 13

10.2. Cláusula MINUS o EXCEPT 14

10.3. Combinación de operaciones 14

10.4. Ordenación de consultas 15

/ 11. Caso práctico 1: “Sentencias multitabla en SQL Live” 15

/ 12. Caso práctico 2: “Subconsultas en Oracle SQL Live” 16

/ 13. Resumen y resolución del caso práctico de la unidad 16

/ 14. Bibliografía 18

OBJETIVOS

Conocer el concepto de consulta multitabla y cómo se elabora.

Entender qué es una subconsulta y cómo debe emplearse.

Aprender sobre el concepto y la utilización de operaciones de conjuntos: unión, intersección y diferencia entre consultas.

Comprender las aplicaciones y usos de las consultas multitabla y las subconsultas.

/ 1. Introducción y contextualización práctica

En temas previos, hemos analizado diversos tipos de consultas a bases de datos, todas ellas centradas en acceder a la información contenida en una única tabla.

No obstante, también hemos aprendido que una de las principales ventajas de las bases de datos es la capacidad de distribuir la información en múltiples tablas para evitar la duplicidad de datos.

Por lo tanto, es evidente que, en numerosas ocasiones, puede ser necesario extraer información de varias tablas en una sola consulta. Esta práctica es bastante común en entornos profesionales de bases de datos.



Fig.1. Podemos utilizar Oracle SQL Developer para practicar consultas y aplicar la teoría de este tema.

En este tema, profundizaremos en cómo llevar a cabo consultas multitabla y exploraremos los desafíos que pueden surgir al realizarlas. Proporcionaremos ejemplos prácticos para facilitar la comprensión de su uso. Posteriormente, nos centraremos en el uso y las características de las subconsultas, también apoyados con ejemplos prácticos.

Como en temas anteriores, la práctica es esencial para dominar este tipo de consultas y aprovechar plenamente las ventajas y el potencial que ofrecen.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Practiquemos consultas multitabla, subconsultas y funciones de conjuntos"

https://on.soundcloud.com/EDDWZridiYGA4UTj8

/ 2. Consultas sobre varias tablas

Las bases de datos relacionales organizan la información en múltiples tablas, las cuales pueden estar interconectadas mediante un campo específico. **El campo utilizado para asociar las tablas suele ser una clave primaria** en una tabla, **que se vincula a una clave externa o foránea** en otra tabla.

El comando SELECT permite **realizar consultas a múltiples tablas en una sola sentencia**, utilizando la operación conocida como **JOIN o composición**. Esto se conoce como **consultas multitabla**.

Las **consultas multitabla** pueden realizarse empleando diversas cláusulas y métodos, y **se dividen en tres** categorías de **composiciones**:

- **Composiciones internas**: Utilizan las cláusulas **INNER JOIN** y **NATURAL JOIN**.
- **Composiciones externas**: Emplean las cláusulas **LEFT JOIN**, **RIGHT JOIN** y **FULL OUTER JOIN**.
- **Composiciones cruzadas** o **productos cartesianos**: Se realizan mediante la cláusula **CROSS JOIN**.

Vídeo 1. "Concepto y ejemplos de uso de sentencias INNER JOIN, LEFT JOIN y RIGHT JOIN"

https://bit.ly/3iz0rOt

Estudiaremos todas ellas en detalle a lo largo de este tema.

De acuerdo con el estándar SQL 99, **la sintaxis** que se puede utilizar **para efectuar consultas multitabla** es la mostrada en la siguiente imagen:

```
SELECT tablaA.columna1, tablaA.columna2, …,
       tablaB.columna1, tablaB.clumna2, …
FROM tablaA
[(INNER) JOIN tablaB USING columna]
[(INNER) JOIN tablaB
           ON (tablaA.colUM = tablaB.colum)]
[NATURAL JOIN tablaB]
[CROSS JOIN tablaB]
[LEFT JOIN tablaB
        ON (tabla.columna = tablaB.columna)]
[RIGHT JOIN tablaB
        ON (tabla.columna = tablaB.columna)]
[FULL OUTER JOIN tablaB
        ON (tabla.columna  = tablaB.columna)]
```

Fig.2. Sintaxis de sentencias multitabla según el estándar SQL 99.

¡! Sabías que...

En las primeras bases de datos relacionales de los años 70, no había una forma sencilla de combinar datos de diferentes tablas. Los programadores tenían que escribir código adicional, lo que complicaba y hacía propenso a errores el proceso.

/ 3. Composiciones internas

En las bases de datos relacionales, a menudo necesitamos combinar datos de diferentes tablas para obtener información completa. Para hacer esto, utilizamos las cláusulas **INNER JOIN** y **NATURAL JOIN**.

3.1. JOIN o INNER JOIN

La cláusula **INNER JOIN**, o simplemente **JOIN**, es una forma de unir dos tablas. **Combina las filas de ambas tablas cuando hay coincidencias en una columna específica. Solo muestra las filas donde los datos coinciden en ambas tablas**.

Como ejemplo, partiremos de las tablas EMPLEADOS y SUCURSALES, donde queremos ver los empleados que tiene una sucursal concreta. Las tablas EMPLEADOS y SUCURSALES tienen una columna llamada COD_SUCURSAL, la cual conecta ambas tablas al contener los mismos datos. Aunque en este caso las columnas tienen el mismo nombre, no es un requisito que sea así. En la siguiente imagen se muestra un ejemplo de consulta que obtendría esta información.

```
SELECT E.*, A.LOCALIDAD
FROM EMPLEADOS E JOIN SUCURSALES A
ON E.COD_SUCURSAL = A.COD_SUCURSAL;
```

Fig.3. Ejemplo de consulta usando JOIN.

3.2. NATURAL JOIN

La cláusula **NATURAL JOIN** también **combina tablas**, pero lo hace **automáticamente basándose en columnas que tienen el mismo nombre y tipo en ambas tablas**. No se necesita especificar las columnas manualmente.

A diferencia de INNER JOIN, en este caso **no es necesario usar la cláusula ON**.

En la imagen mostrada a continuación, se muestra un ejemplo de utilización de la cláusula NATURAL JOIN usando las tablas EMPLEADOS y SUCURSALES del ejemplo anterior:

```
SELECT *
FROM EMPLEADOS
NATURAL JOIN SUCURSALES;
```

Fig.4. Ejemplo de consulta usando NATURAL JOIN.

En el ejemplo anterior, la cláusula NATURAL JOIN combinaría automáticamente las tablas EMPLEADOS y SUCURSALES usando la columna COD_SUCURSAL.

Enlaces de interés...

En el este enlace encontrarás un vídeo donde podrás observar varios ejemplos de sentencias SQL en los que se realizan consultas sobre varias tablas de la base de datos utilizando la cláusula INNER JOIN.

/ 4. Producto cartesiano

El producto cartesiano o producto cruzado se caracteriza por asociar cada fila de una tabla a todas las filas de otra. Dicho de otra forma, **es el resultado de realizar todas las combinaciones posibles entre las filas de las tablas implicadas en la consulta**.

Para realizar esta combinación, existen dos formas de indicarlo en una consulta SQL:

- **Con la cláusula CROSS JOIN**: Es necesario indicar, en la cláusula **FROM** de la consulta, todas las tablas desde las cuales se quiere extraer la información separándolas entre sí con la cláusula **CROSS JOIN**.
- **Sin cláusulas explícitas**: Es necesario indicar, en la cláusula **FROM** de la consulta, todas las tablas desde las cuales se quiere extraer la información separándolas con una coma (,).

En la siguiente imagen, se puede ver un ejemplo de consulta que realizaría un producto cartesiano sobre las tablas TAREAS y EMPLEADOS sin utilizar cláusulas.

```
SELECT cod_tarea, dni_empleado, nombre_empleado
FROM TAREAS, EMPLEADOS;
```

Fig.5. Ejemplo de consulta producto cartesiano sin cláusulas.

En esta otra imagen se puede ver el ejemplo equivalente utilizando la cláusula CROSS JOIN.

```
SELECT cod_tarea, dni_empleado, nombre_empleado
FROM TAREAS CROSS JOIN EMPLEADOS;
```

Fig.6. Ejemplo de consulta producto cartesiano con CROSS JOIN.

Esta consulta generaría como resultado una tabla similar a la siguiente, en la que se mostrarían todas y cada una de las tareas asociándolas a cada uno de los empleados.

cod_tarea	dni_empleado	nombre_empleado
101	1	Alice
101	2	Bob
102	1	Alice
102	2	Bob

Tabla 1. Tabla de resultados de un CROSS JOIN.

/ 5. Composiciones externas

Están formadas por las cláusulas **LEFT JOIN**, **RIGHT JOIN** y **FULL OUTER JOIN**. Se utilizan para **combinar filas de dos tablas y también incluyen las filas que no tienen coincidencias en una de las tablas**. Son similares a la cláusula INNER JOIN, con la salvedad de que en el resultado obtenido se añaden filas que no cumplen con la condición de combinación.

5.1. LEFT JOIN y NATURAL LEFT JOIN

La cláusula **LEFT JOIN toma todas las filas de la tabla de la izquierda y las combina con las filas coincidentes de la tabla de la derecha**. Si no hay coincidencias, muestra las filas de la izquierda y pone NULL en las columnas de la derecha.

En la siguiente imagen, se muestra un ejemplo del uso de LEFT JOIN:

```
SELECT emp.nombre, emp.id_sucursal, suc.nombre
FROM empleados emp
LEFT JOIN sucursales suc
ON emp.id_sucursal = suc.id_sucursal;
```

Fig.7. Ejemplo de consulta con LEFT JOIN.

Esta consulta muestra los nombres y los id_sucursal de todos los empleados, y los nombres de las sucursales de la tabla derecha (sucursales) cuyo id_sucursal coincida con el de la tabla izquierda (empleados).

Existe también la cláusula **NATURAL LEFT JOIN**, que hace lo mismo que LEFT JOIN, pero automatizando el proceso de combinación basándose en las columnas con el mismo nombre.

5.2. RIGHT JOIN y NATURAL RIGHT JOIN

La cláusula **RIGHT JOIN** hace lo opuesto a LEFT JOIN: **toma todas las filas de la tabla de la derecha y las combina con las filas coincidentes de la tabla de la izquierda**. Si no hay coincidencias, muestra las columnas de todas las filas de la tabla derecha y pone NULL en las columnas de la tabla izquierda.

En la siguiente imagen, se muestra un ejemplo del uso de RIGHT JOIN.

```
SELECT emp.nombre, emp.id_sucursal, suc.nombre
FROM empleados emp
RIGHT JOIN sucursales suc
ON emp.id_sucursal = suc.id_sucursal;
```

Fig.8. Ejemplo de consulta con RIGHT JOIN.

Esta consulta muestra todas las sucursales y sus empleados. Si alguna sucursal no tiene empleados, todavía aparecerá en el resultado, pero con NULL en los datos del empleado.

Existe también la cláusula **NATURAL RIGTH JOIN**, que actúa de la misma forma que RIGHT JOIN, pero **une las tablas automáticamente basándose en las columnas con el mismo nombre**.

/ 6. FULL OUTER JOIN

Esta cláusula **combina todas las filas de ambas tablas**. **Muestra las filas con coincidencias y** también **las filas sin coincidencias**, poniendo **NULL donde no haya datos**.

En la siguiente imagen, se muestra un ejemplo del uso de FULL OUTER JOIN.

```
SELECT *
FROM empleados
FULL OUTER JOIN sucursales
ON empleados.id_sucursal = sucursales.id_sucursal;
```

Fig.9. Ejemplo de consulta con FULL OUTER JOIN.

Esta consulta muestra todos los empleados y todas las sucursales. Si algún empleado no tiene sucursal o alguna sucursal no tiene empleados, ambos aparecerán en el resultado con NULL en las columnas donde no haya datos.

Como resumen de todas las composiciones internas vistas, se muestra, en la próxima imagen, de una forma gráfica, la diferencia entre los distintos tipos de composiciones existentes en SQL:

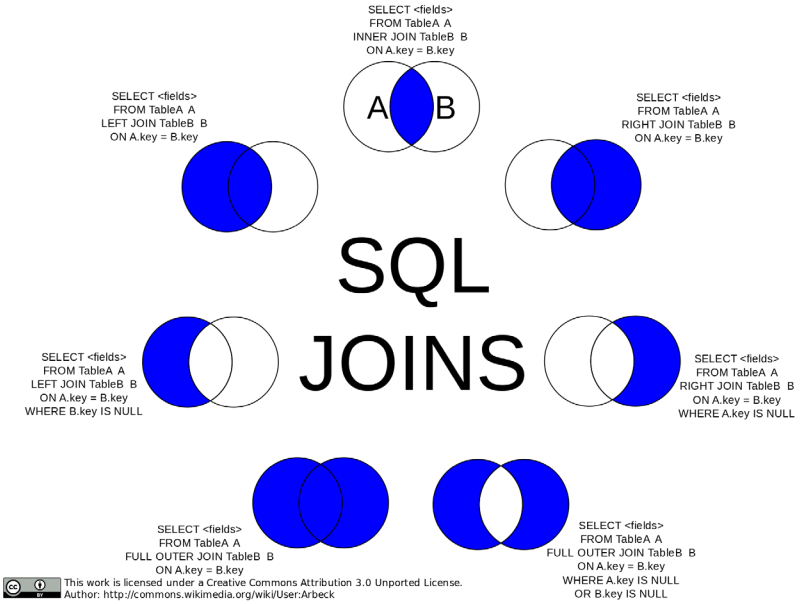


Fig.10. Tipos de JOINS o composiciones en SQL.

En la imagen, se puede ver que la composición interna INNER JOIN combina únicamente los elementos que son comunes en las tablas A y B.

En cambio, las composiciones externas (LEFT JOIN, RIGHT JOIN y FULL OUTER JOIN) pueden combinar todos los elementos de una tabla, incluyendo los comunes con la otra tabla o solo los elementos que no coinciden. Dependiendo del resultado deseado, también es posible mostrar todos los elementos de ambas tablas o solo aquellos que no son comunes a ambas.

/ 7. Subconsultas

Es una consulta incrustada dentro de otra consulta. Esto significa que los resultados obtenidos de la subconsulta se utilizan para realizar una nueva consulta, todo en una única sentencia SQL.

Normalmente, las subconsultas tienen una **sintaxis** similar a la que se muestra a continuación en la imagen. Se puede ver, en color azul, lo que se denomina como **SUBSELECT** o **subconsulta**. Para evitar errores, la subconsulta debe escribirse entre paréntesis:

```
SELECT Columnas
FROM tabla
WHERE expresión [OPERADOR]
      (SELECT colums FROM tabla);
```

Fig.11. Ejemplo de sintaxis.

En las subconsultas, podemos usar **operadores de comparación** como '**<**', '**>**', '**<=**', '**>=**', '**!=**', '**=**'. Pero estos solo pueden utilizarse en subconsultas escalares, que devuelven un único valor.

En esta otra imagen se puede ver un ejemplo de una subconsulta para obtener los empleados que tengan menor salario que 'Martina'.

```
SELECT nombre_empleado, salario
FROM EMPLEADOS
WHERE salario < ( SELECT salario
                  FROM EMPLEADOS
                  WHERE nombre_empleado='Martina');
```

Fig.12. Ejemplo de subconsulta.

Podemos distinguir **tres tipos de subconsultas**:

- **Subconsultas de fila**: Devuelven varias columnas, pero en una única fila.
- **Subconsultas de tabla**: Pueden devolver de 0 a varias filas y estas estarán conformadas por una o varias columnas.
- **Subconsultas escalares**: Su resultado está formado por una fila y una columna.

Enlaces de interés...

En este enlace encontrarás un vídeo en el que podrás observar varios ejemplos de sentencias SQL en los que se utilizan subconsultas para filtrar la información almacenada en la base de datos.

7.1. Subconsultas de varias filas

Hemos visto que solo podemos utilizar operadores de comparación con subconsultas que devuelvan una fila. Para subconsultas que devuelvan varias filas, debemos emplear las siguientes instrucciones:

INSTRUCCIÓN	USO
ANY/SOME	Son intercambiables. La condición es verdadera si se cumple para al menos un valor del conjunto.
ALL	La condición es verdadera si se cumple para todos los valores en el conjunto de resultados.
IN	Se cumple si un valor está contenido entre los resultados de la subconsulta.
NOT IN	Es la negación del anterior. Se cumple si el valor no está en el conjunto de resultados.

Tabla 2. Instrucciones para subconsultas de varias filas.

/ 8. Subconsultas con ALL, ANY, IN y NOT IN

Las cláusulas **ALL**, **ANY**, **IN** y **NOT IN** en SQL permiten realizar comparaciones avanzadas con conjuntos de datos, ayudando a crear consultas más precisas.

8.1. ALL y ANY

Se utilizan en combinación con los operadores de comparación. Permiten comparar una expresión o valor con los valores devueltos por una subconsulta. **Al comparar con ALL todos los registros de la subconsulta deben cumplir la condición del operador**. En la siguiente imagen, podemos ver un ejemplo del uso de ALL para obtener los empleados con mayor salario.

```
SELECT nombre, salario
FROM EMPLEADOS
WHERE salario >= ALL ( SELECT salario
                       FROM EMPLEADOS );
```

Fig.13. Ejemplo de consulta con la cláusula ALL.

El uso de **ANY implica que alguno de los registros de la subconsulta debe cumplir la condición del operador, pero no todos**. En la próxima imagen, se muestra un ejemplo del uso de ANY para obtener los empleados que tengan mayor salario que el empleado 5.

```
SELECT NOMBRE, APELLIDOS, SALARIO
FROM  EMPLEADOS
WHERE SALARIO > ANY ( SELECT SALARIO
                      FROM EMPLEADOS
                      WHERE ID_EMPLEADO = 5);
```

Fig.14. Ejemplo de consulta con la cláusula ANY.

8.2. IN y NOT IN

Trabajan directamente con un conjunto de resultados para comprobar si el valor o expresión que queremos comparar está o no dentro del conjunto.

La cláusula **IN** se cumple cuando el valor o expresión está dentro del conjunto de resultados. En la imagen, se muestra una consulta que devuelve los empleados que también son clientes:

```
SELECT NOMBRE, APELLIDOS
FROM  EMPLEADOS
WHERE DNI IN (SELECT DNI FROM CLIENTES);
```

Fig.15. Ejemplo de subconsulta con IN.

Por el contrario, la cláusula **NOT IN** se cumple cuando el valor no está en el conjunto de resultados. En la siguiente imagen, se muestra una consulta que devuelve los empleados que no son clientes.

```
SELECT NOMBRE, APELLIDOS
FROM  EMPLEADOS
WHERE DNI NOT IN (SELECT DNI FROM CLIENTES);
```

Fig.16. Ejemplo de subconsulta con NOT IN.

/ 9. Subconsultas con EXISTS, NOT EXISTS, FROM, JOIN, GROUP BY y HAVING

En SQL, las subconsultas se pueden utilizar con las cláusulas **EXISTS**, **NOT EXISTS**, **FROM**, **JOIN**, **GROUP BY** y **HAVING** para realizar consultas complejas que dependen de condiciones específicas y agrupaciones de datos.

9.1. Subconsultas con EXISTS y NOT EXISTS

La cláusula **EXISTS** devuelve '**verdadero**' **si la subconsulta interna retorna algún valor; de lo contrario, devuelve 'falso'**. También se puede utilizar en su forma negativa: **NOT EXISTS**.

En la siguiente imagen, se muestra un ejemplo con los productos que están presentes en la tabla ALMACEN, utilizando consultas correlacionadas (incluyen alias de una tabla específica):

```
SELECT NOMBRE, PRECIO, FABRICANTE
FROM  PRODUCTOS P
WHERE EXISTS (SELECT NOMBRE, PRECIO
              FROM ALMACEN
              WHERE NOMBRE = P.NOMBRE
                AND PRECIO = P.PRECIO);
```

Fig.17. Subconsulta utilizando ALL.

9.2. Subconsultas con FROM y JOIN

Las subconsultas se pueden anidar con otras cláusulas, como **FROM** o **JOIN**, con el objetivo de obtener datos necesarios para la consulta principal. En las siguientes imágenes, podemos ver unos ejemplos de consultas para conocer información sobre empleados con el cargo de responsables que tengan un salario específico.

```
SELECT E.NOMBRE
FROM EMPLEADOS E,
    (SELECT NOMBRE, ID_EMPLEADO
    FROM EMPLEADOS
    WHERE SALARIO > 1000) EMP2
WHERE E.RESPONSABLE = EMP2.ID_EMPLEADO;
```

Fig.18. Ejemplo de subconsulta con FROM.

```
SELECT E.NOMBRE
FROM EMPLEADOS E
JOIN (SELECT NOMBRE, ID_EMPLEADO
        FROM EMPLEADOS
        WHERE SALARIO > 1000) EMP2
ON E.RESPONSABLE = EMP2.ID_EMPLEADO;
```

Fig.19. Subconsulta con JOIN.

9.3. Subconsultas con GROUP BY y HAVING

Las subconsultas también pueden combinarse con las cláusulas **GROUP BY** y **HAVING**. En el ejemplo de la imagen que se muestra a continuación, se selecciona el departamento cuya media de salarios es mayor que la media total:

```
SELECT DEPARTAMENTO,
FROM EMPLEADOS GROUP BY DEPARTAMENTO
HAVING AVG(SALARIO) >
        (SELECT AVG(SALARIO) FROM EMPLEADOS);
```

Fig.20. Subconsulta con HAVING.

/ 10. Operaciones de conjuntos

A menudo surge la necesidad de **unir, encontrar intersecciones y/o identificar diferencias, entre datos en dos tablas** que contienen distintos registros del mismo tipo de datos, **para crear una tabla única**. Para lograr esto, se utilizan las siguientes cláusulas:

- **UNION**.
- **INTERSECT**.
- **MINUS**.

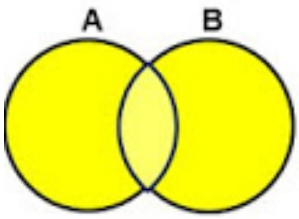
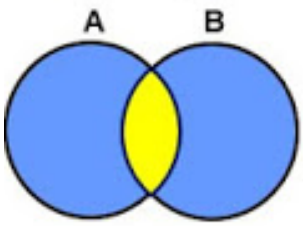
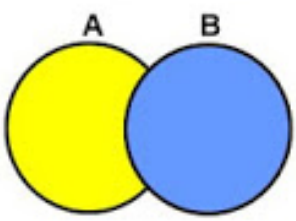
UNIÓN	INTERSECCIÓN	DIFERENCIA
		

Tabla 3. Representación gráfica de UNION, INTERSECT y MINUS en el ámbito de conjuntos de datos.

Estas cláusulas **combinan los resultados de dos sentencias SELECT**. **Es esencial que el orden, el tipo y el número de columnas sean idénticos**. Por ejemplo, no se pueden mezclar columnas de tipo NUMBER con columnas de tipo DATE. Además, estas cláusulas **pueden utilizarse de forma anidada**.

10.1. Cláusula UNION e INTERSECT

La cláusula **UNION combina los resultados de dos consultas SELECT, eliminando las filas duplicadas**. Si se desea **mantener las filas duplicadas**, se puede utilizar **UNION ALL**.

Audio 1. "Características, ventajas y desventajas de UNION ALL"
https://on.soundcloud.com/roJmh2F1ssvSqNpi7

En la siguiente imagen, se muestra un ejemplo de unión de consultas:

```
SELECT NOMBRE, APELLIDOS, DNI FROM CLIENTES
UNION
SELECT NOMBRE, APELLIDOS, DNI FROM SOCIOS;
```

Fig.21. Ejemplo de unión de consultas.

La cláusula **INTERSECT**, por su parte, **devuelve solo las filas que están presentes en dos consultas SELECT**, también **eliminando las filas duplicadas**.

En esta otra imagen, se muestra un ejemplo de intersección de consultas:

```
SELECT TIPO, MODELO FROM PIEZAS
WHERE ALMACEN = 1
INTERSECT
SELECT TIPO, MODELO FROM PIEZAS
WHERE ALMACEN = 2;
```

Fig.22. Ejemplo de intersección de consultas que devuelve las piezas que están en ambos almacenes.

10.2. Cláusula MINUS o EXCEPT

La cláusula **MINUS o EXCEPT permite obtener las filas presentes en el resultado de la primera consulta SELECT, pero que no están en el resultado de la segunda**. Las filas duplicadas se descartan en el proceso. La cláusula **EXCEPT** es la versión alternativa utilizada en otras versiones de SQL y en otros SGBD.

En la siguiente imagen, podemos ver un ejemplo de uso de la cláusula MINUS para obtener las piezas que están solo en el almacén 1.

```
SELECT TIPO, MODELO FROM PIEZAS
WHERE ALMACEN = 1
MINUS
SELECT TIPO, MODELO FROM PIEZAS
WHERE ALMACEN = 2;
```

Fig.23. Ejemplo de uso de la cláusula MINUS.

10.3. Combinación de operaciones

Como ya se ha mencionado anteriormente, estas operaciones de conjuntos pueden anidarse para formar consultas de mayor complejidad.

Para mayor claridad y con el fin de evitar errores, **es recomendable usar paréntesis en las consultas resultantes de estos anidamientos**.

En la siguiente imagen, se muestra una consulta anidada para obtener los elementos de los talleres 1 y 2 que no estén en el 3:

```
(SELECT FABRICANTE, MODELO FROM  MECANICA
WHERE TALLER = 1
UNION
SELECT FABRICANTE, MODELO FROM  MECANICA
WHERE TALLER = 2)
MINUS
SELECT FABRICANTE, MODELO FROM  MECANICA
WHERE TALLER = 3;
```

Fig.24. Ejemplo de combinación de operaciones.

10.4. Ordenación de consultas

Para ordenar los datos resultantes de utilizar las cláusulas UNION, INTERSECT o MINUS, se puede utilizar la cláusula ORDER BY. Debe tenerse en cuenta que esta debe hacer referencia siempre a las columnas del primer SELECT.

```
SELECT TELEFONO1 FROM EMPLEADOS
UNION
SELECT TELEFONO2 FROM EMPLEADOS
ORDER BY TELEFONO1;
```

Fig.25. Ejemplo de ordenación de consultas con UNION.

/ 11. Caso práctico 1: "Sentencias multitabla en SQL Live"

Planteamiento: Para adquirir un buen conocimiento y una correcta comprensión de las sentencias SQL, es fundamental practicar con ellas. Por lo tanto, utilizaremos la herramienta Oracle Live SQL, que hemos revisado en temas anteriores, para hacer estas prácticas.

Nudo: Para realizar esta práctica, usaremos las instrucciones aprendidas en temas anteriores para definir las siguientes tablas:

PRODUCTOS	CATEGORÍAS
ID_PRODUCTO	ID_CATEGORÍA
NOMBRE_PRODUCTO	NOMBRE_CATEGORÍA
ID_CATEGORÍA	
PRECIO	

Tabla 4. Ejemplos de tablas a crear para las consultas.

La meta en esta actividad es practicar el uso de las consultas multitabla estudiadas en este tema, por lo que se deben utilizar varias consultas tipo JOIN. Por eso es necesario insertar, en cada una de estas tablas, al menos, 5 elementos. Se debe tener en cuenta que puede (y debe) haber categorías que no tengan productos asociados.

Debe especificarse el objetivo de la consulta y explicar el resultado obtenido.

Desenlace: La siguiente consulta es una de las que se podrían desarrollar. En ella se devuelven los nombres de todos los productos que tengan asociada una categoría y el nombre de esta.

Una posible consulta puede ser la siguiente, en la que se devuelven los nombres de producto y de categoría de ambas tablas a través de una cláusula INNER JOIN, es decir, solo los que son comunes a ambas tablas.

```
SELECT NOMBRE_PRODUCTO, NOMBRE_CATEGORIA
FROM  PRODUCTOS P
INNER JOIN CATEGORIAS C
  ON P.ID_PRODUCTO = C.ID_PRODUCTO;
```

Fig.26. Ejemplo de composición interna sobre las tablas indicadas.

/ 12. Caso práctico 2: "Subconsultas en Oracle SQL Live"

Planteamiento: Continuaremos con la práctica del uso de las sentencias SQL vistas en este tema para reforzar los conocimientos adquiridos. Haremos uso, para ello, de Oracle SQL Live.

Nudo: Además de las tablas que hemos creado en el caso práctico 1, vamos a crear las siguientes nuevas tablas:

EMPLEADOS	DEPARTAMENTOS
ID_EMPLEADO	ID_DPTO
NOMBRE	NOMBRE
APELLIDOS	RESPONSABLE DPTO.
TELÉFONO	
SALARIO	
RESPONSABLE	
ID_DPTO	

Tabla 5. Tabla de ejemplo para practicar subconsultas.

Sobre estas tablas debemos insertar varios registros, ya que la finalidad de esta actividad es poner en práctica las subconsultas estudiadas durante el tema, de forma que se utilizan diferentes cláusulas y variantes.

Debe indicarse la finalidad de la consulta y exponer el resultado obtenido.

Desenlace: Uno de los posibles ejemplos de subconsultas a desarrollar podría ser el que se muestra a continuación:

```
SELECT DEPARTAMENTO,
FROM EMPLEADOS
GROUP BY DEPARTAMENTO
HAVING AVG(SALARIO) >
    (SELECT AVG(SALARIO) FROM EMPLEADOS);
```

Fig.27. Ejemplo de subconsultas sobre las tablas indicadas.

/ 13. Resumen y resolución del caso práctico de la unidad

En este tema hemos visto las diferentes cláusulas que nos ofrece SQL para realizar consultas que impliquen el uso de varias tablas relacionadas entre sí, algo crucial en las bases de datos relacionales.

Para continuar, hemos visto también las posibilidades existentes para el anidamiento de consultas.

Por último, hemos conocido las operaciones de conjuntos que se pueden aplicar sobre los resultados.

A continuación podrás ver el esquema resumen del tema:

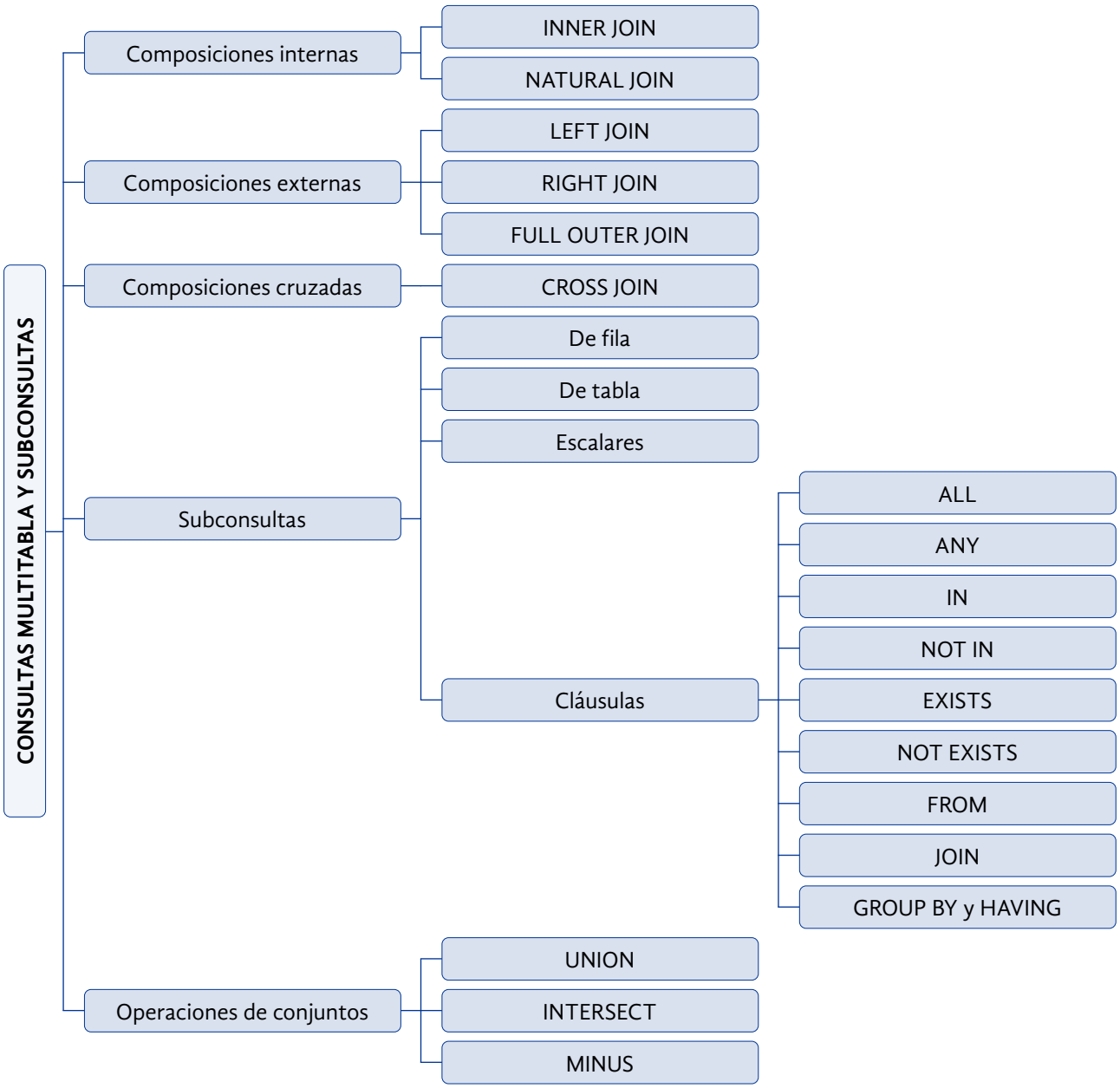


Fig.28. Esquema resumen del tema.

Resolución del caso práctico de la unidad

En el caso práctico planteado en el audio de introducción del tema, se exponía la necesidad de nuestro amigo de extraer el nombre y los apellidos de todos los empleados y de todos los clientes en una única consulta.

En la siguiente imagen, se puede ver un ejemplo de consulta que nos permitiría obtener toda esta información.

```
SELECT NOMBRE, APELLIDOS
FROM EMPLEADOS
UNION
SELECT NOMBRE, APELLIDOS
FROM CLIENTES;
```

Fig.29. Ejemplo de consulta para obtención de la información.

/ 14. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, Iván y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.